



AI JOB AUTOMATION TOOL

Technical Document for review and guide



MARCH 23, 2026

DIPIN KHATRI

Contents

Prerequisites	1
1. Project Overview	2
1.1 What it does	2
1.2 Tech Stack	2
2. Setup & Installation	2
2.1 prerequisites	2
2.2 Environment Variables	3
2.3 Setup your docker files	3
3. System Architecture	3
3.1 Setting up Index.js	3
3.3 Migrate.js	5
3.4 Db.js	5
4. Config Layer	5
4.1 Redis.js	5
4.2 Logger.js	6
5. Jobs	6
5.1 Cron	6
6. Controllers	7
6.1 Application Engine Controller	7
6.2 Apply Controller	7
6.3 Auth Controller	8
6.4 Document Controller	8
6.5 Github Controller	9
6.6 Job Controller	9
6.7 Matching Controller	10
6.7 Resume Controller	10
7. Services	11
7.1 Application Engine Service	11
7.2 Application Service	11

7.3 Apply Service	12
7.4 Document Service	12
7.5 Job Service.....	13
7.6 Matching Service.....	13
7.7 Notification Service	14
7.8 Pdf Service.....	15
7.9 Pipeline Service	15
7.10 Playwright Agent.....	15
8. Middleware	16
8.1 Auth Middleware	16
8.2 Rate Limiter Middleware	16
9. Scripts.....	17
9.1 FetchRealJobs.....	17
10. Routes	17
10.1 Application Engine Route	17
10.2 Apply Routes	17
10.3 Auth Routes.....	18
10.4 Document Routes	18
10.5 GitHub Routes	18
10.6 Job Routes.....	18
10.7 Matching Routes	19
10.8 Pipeline Routes	19
10.9 Resume Routes	19
11. Client	19
11. Database	20
12. Key Design Decision	22
12.1 Weighted Scoring over Strict Threshold	22
12.2 Jsearch over multiple free api.....	22
12.3 Two-Pass PDF System	22
12.4 SSE for Live Pipeline Progress	22
12.5 Redis Retry Queue	23
12.6 Separate JWT Secrets.....	23
12.7 Semi-Automated Playwright.....	23

12.8 PostgreSQL Vs. MongoDB	23
References	24
1. Backend & Runtime	24
2. Frontend	24
3. Database & Caching	24
4. Authentication & Security.....	24
5. PDF Processing.....	24
6. Browser Automation.....	25
7. External APIs & Services.....	25
8. Web Standards & Protocols	25

Prerequisites

Install these following things:

1. Docker
2. Node.js
3. Express
4. React.js
5. Cors
6. PdfParser
7. PdfKit
8. Redis
9. PostGreSQL
10. JWT (jsonwebtoken)
11. Bcrypt
12. Resend
13. Axios
14. Playwright
15. Multer
16. Winston (logging)
17. Node-cron
18. Express-rate-limit
19. Vite/ Tailwind
20. API keys (Anthropic, Adzuna, OpenWebNinja, Resend)

Create your folder structure

Client - All frontend

Backend:

1. Routes – all the api routes
2. Services – job service, matching services etc.
3. DB – migrations, sql queries
4. Controllers – the logics of auth, db, services
5. Scripts – fetching jobs
6. Middleware – auth and redis rate limiter to hit api
7. Jobs - cronJobs
8. Config – redis config and logger

Tools Used

Backend	Database	AI	Frontend
Node.js + Express	PostGreSQL + Redis	Anthropic	React + Vite + Tailwind

1. Project Overview

This is a full-stack automated job application system that uses Anthropic's Claude Ai to find, match, generate tailored applications for job seekers and semi automated applying system integrated. The system runs automatically on schedule – finding relevant jobs, with a good scoring system and, generating a unique resume and cover letter for each matching positions. Automatically applies to direct company where as one's with captcha and bots are needed to fill in manually.

1.1 What it does

- Fetches real job listing from Adzuna API and OpenWebNinja
- Uses Anthropic AI to score each jobs against your skills, experience and job preference
- Automatically applies to job above threshold and rejects jobs below threshold, some need to manually apply
- Generates unique tailored resume and cover letter for each job
- Uses two pass self correcting PDF system to ensure perfect 1-page output
- Caches results in Redis and runs on scheduled cron job

1.2 Tech Stack

Layer	Technology	Purpose
Backend	Node.js+ Express	REST API
Database	POSTGRESQL + Docker	Storing primary data
Cache	Redis+Docker	Job caching, retry queue, session data
AI	Anthropic Claude AI	Job matching, resume/cover letter, Auto apply
Auth	JWT+ bcrypt	User authentication, refresh token
PDF	PDFparser+ pdftkit	Parse cv and resume and cover-letter generation
Frontend	React + Vite + Tailwind	Dashboard UI
Scheduler	Node-cron	Automated job fetching and pipeline runs

2. Setup & Installation

2.1 prerequisites

- Node.js
- Docker Desktop(for redis and postgresql)
- Anthropic api key
- Adzuna, OpenWebNinja API key
- Resemd API key

2.2 Environment Variables

DB_USER= #put your own user, DB_PASSWORD= #put your own password, DB_NAME= #put your own, REDIS_URL= #put your own redis, PORT= #put your own port, JWT_SECRET= #put your own jwt, DATABASE_URL= #put your own database, ANTHROPIC_API= #put your api call here, ADZUNA_APP_ID= #put your_app_id_here, ADZUNA_APP_KEY= #put your_app_key_here, CLIENT_URL= #put your frontend link here,JWT_REFRESH_TOKEN=#put your refresh token here

2.3 Setup your docker files

- Docker-compose.yml
 - Multiple services like redis, postgre work together
 - Volume is the persistence db storage
- Dockerfile
 - Base image(blueprint)
 - All dependencies and source code locations
 - Commands to run our app CMD [“node”, “src/index.js”]

Docker is a **container** that packages your app and its environment, allowing it to run anywhere with the same behavior. For this project:

- docker-compose.yml tells Docker which services to start and how they connect.
- Dockerfile **containerizes the application**, providing a blueprint that can run anywhere in the world.

To run docker :

- docker compose up -d(detached mode) runs in background else it runs in the terminal

To stop docker

- docker compose down (shuts down)

3. System Architecture

3.1 Setting up Index.js

This is the main entry point of the application. It initializes middleware, environment variables and routes all API requests.

3.2 Database Layer

ID	Service	What Happens
1	Redis	Using it for caching
2	Postgre SQL	Relational Database for sql querying

PostgreSQL is a relational database. User will have resume link to applications, applications link to jobs. Relational data fits SQL better. PostgreSQL also have better support for array types (skills, required, matched_skills) which are used extensively.

Inside our db folder it has migrations folder along with db.js and migration.js

Contains Migration folder with sql queries

File	Purpose/What happens
001_initial_schema	Creates the initial tables if they don't exist, using UUIDs (uuid_generate_v4) for primary keys
002_add_columns	Alters existing tables to add job_search, country, and is_active columns
003_refresh_tokens	Creates a table to store refresh tokens for authentication, which expire after 1 hour
004_update_jobs_table	Updates tables like jobs, resume, coverletter, and applications; drops and recreates tables where necessary
005_update_applications	Adds columns for matched_skills, missing_skills, and match_reasoning in the applications table
006_update_document_table	Deletes old ai_resume and coverletter tables and creates new ones
007_add_pdf_columns	Updates ai_resume column to support PDF in Base64 format
008_add_user_profile	Updates the users table to add additional profile information
009_add_user_prefs	Updates the users table to add preferred companies and blocked companies
010_rename_skills_required	Rename skills_required to required_skills in the table for consistency
011_update_application_status	Adds new status values to the application_status enum — auto_applied, manual_required, manually_applied, rejected, interviewing. Also adds tracking columns to the applications table — apply_method (auto or manual), apply_attempted_at (timestamp of apply attempt), apply_error (stores error reason if apply failed), notification_sent (tracks whether the email notification was sent)
012_update_application_table	Adds cover_letter_path and notes columns to the applications table — cover_letter_path stores the path to the generated cover letter, notes stores any error messages or manual notes about the application

013_updated_resume_path	ALTER TABLE applications ADD COLUMN IF NOT EXISTS resume_path text;
014_default_job_search_status	ALTER TABLE applications ADD COLUMN IF NOT EXISTS resume_path text;

3.3 Migrate.js

- Creates a migrations table to track executed scripts
- Reads .sql files in the folder in order
- Checks which migrations have already run
- Executes new migrations only and marks them complete

Flow of this is

- Load the environment
- Create Migration table if not exists
- Store all executed sql in migration table with timestamp
- Store them in set for fast lookup
- If Set contains them skip else
- Execute the remaining sql files
- Release the pool and end connection done once at the start

3.4 Db.js

- Provides a shared db connection for our entire application
- Loads environment – pool remains open – exports the pool

4. Config Layer

This folder centralizes all configuration and setup for external tools and services such as redis.

Redis – is an in-memory key value storage, which is faster to access and temporary database that lives in RAM instead of disk. Common use case of redis are:

- Caching – store the result of expensive query as to not hit db all the time
- Rate Limiting – track how many requests a user has made in a time window
- Job Queues – pass task to background workers
- Sessions – store user session data

This folder contains 2 files:

- Redis.js
- Logger.js

4.1 Redis.js

- Connects to redis once. Creates the Redis client only once. If it's already created we return to avoid multiple connections.

- Socket is a reconnection strategy where if the connection drops it retries up to 10 times. Each retry has 3second max delay capped at 3000ms. After 10 fails, gives up and throws error.
- The Proxy wraps that so you can just do `redis.get(key)` – it handles the `await` internally before calling the method otherwise we'd have to `await for getRedisClient().get(key)`

Socket is like a pipe connecting Redis and our app. If that pipe breaks, it tells the app what to do.

Proxy is used for convenience as it hides the complexity from every other file in our app, kind of like a middleman, so we never have to manually wait for a connection.

4.2 Logger.js

This uses a logging library called Winston and Winston gives us levels, formatting and transports our logs. We have three methods:

- Levels - it checks our node environment, in production logs only info and avoids noise and in development logs everything, including debug for full visibility along with noise(useless data)
- Format – shapes every log into [timestamp] Level: message, and if it's an error, it also attaches the stack trace so you know where in the code it broke
- Transport -sends logs to 3 places at once: console, `error.log(errors only)` and `combined.log(everything)`

Note: The same error can be present in both files since `combine.log` has everything.

5. Jobs

The jobs folder holds background tasks and code that runs automatically on a schedule, not triggered by the user request. Kind of like an automated worker that does thing behind the scene.

5.1 Cron

We use cron which is a scheduler. It lets us run a function automatically at specific times, like an alarm clock for our code. This has three functional parts

- Schedule: Defines the schedule it should run
 - Runs across all 7 days period
 - 8am/10am/1pm/3pm on weekdays (Jobs are posted)
 - 1 time on weekend(9am)
 - 0-Sunday through 6-Saturday
 - Cron uses 24 hour clock so 08 – 8am | 013 – 1pm
- RunJobFetch
 - Fetches active users
 - Skip if no users
 - Iterates through each users – using `pipelineService.run()` for every user
- StartCronJobs
 - This is what we call when our app boots up
 - Runs our `runJobFetch` using the schedule with timezone of Toronto

Note: If you are somewhere else, you have to change that accordingly to run the schedule at the right time.

6. Controllers

The controllers folder is the entry point for every request. This layer sits between the user and or app's logic. It doesn't contain business logic. Think of it as a waiter which takes your order and passes it to the kitchen to bring back the food. There are 8 controller in controllers. They are:

1. Application Engine Controller
2. Apply Controller
3. Auth Controller
4. Document Controller
5. Github Controller
6. Job Controller
7. Matching Controller
8. Resume Controller

6.1 Application Engine Controller

This controller manages the job application engine where it runs the pipeline, retries failed jobs and updates the user status. Mainly like a control panel for our application automation. The main 3 function in this are:

- Runpipeline
 - Gets the logged in user's Id from the request
 - Calls runPipelineForUser() from service which does the actual work
 - If pipeline was skipped, returns us why with the log
 - If it ran, returns us results(matched, applied, failed, favourites)
- RunRetryQueue
 - Calls processRetryQueue() from the service
 - This mainly triggers retry for any jobs that previously failed
 - Returns us how many were processed
- setStatus
 - Gets the user's id and the new status form req.body
 - Validates the status of one of: active, paused, interviewing, employed
 - If employed sets is_active to false (kind of killswitch to stop pipeline from running saving our cost in ai)
 - Everything is set to true
 - Updated the user in DB through service

6.2 Apply Controller

This controller manages auto-apply pipeline which is semi automated. It handles everything related to applying to jobs. By semi automated meaning if we run into bot detection, captcha which is detected it should be manually applied. There are 4 major functions which are:

- runAutoApply

- checks the min score and limit from the request
- doesn't apply to jobs below the certain score or set limit
- applySingleJob
 - Checks the job exists
 - Checks an application record exists for that user
 - Calls the service processApplication and returns if applied or needs manual intervention
- markAsManualApplied
 - User applied somewhere manually just updates the record to reflect that
- getApplyStatus
 - Queries the DB and returns count of application statuses
 - Like Autoapplied, manual required, rejected etc.

6.3 Auth Controller

This controller manages authentication and user profile. It Registers, logs in and logs out as well as refreshes token using JWT auth. This has 6 function overall which are as follows:

- Register
 - Checks if email already exists, if it does rejects it
 - Hashes password using library called bcrypt
 - Creates the user in DB
- Login
 - Finds the user by email
 - Compares the password against hashed one in Database
 - Generates two tokens – access token(1hr) and refresh token(7 days)
 - Refresh token – used to get new access token
 - Access token – proves who the user is on every request
 - Saves refresh token to db and sends it as httponly cookie
 - Returns access token to the client
- Refresh
 - Reads the refresh token from the cookie
 - Checks if it exists in db
 - Issues new access token so user stays logged in without reentering their password
- Logout
 - Deletes the refresh token from db
 - Clears the cookies
- GetProfile
 - Returns all the logged in user's profile from the db
- UpdateProfile
 - Takes the updated fields from the request body
 - Updates it in the db and returns updated profile

6.4 Document Controller

This controller manages AI generated documents specifically cover letter and resume tailored for specific jobs. This contains 5 functions which are given below:

- generateResumeForJob
 - calls the service to generate a tailored resume for a specific job and returns content
- generateCoverLetterForJob
 - same but for cover letter
- generateBoth
 - generates both resume and coverletter in one call instead of hitting two separate endpoints
- downloadResume
 - fetches the resume from db
 - converts to pdf_base64 back into pdf buffer
 - sends as downloadable pdf file
- downloadCoverLetter
 - similar but for cover letter

6.5 Github Controller

This controller is to connect to user's github profile for ai to analyse and extract the technical skills from their public repo.

- Extract the username
 - Extract from url endpoint github.com/yourname - youname
- Fetch repos from github API
 - Gets the user's 20 most recently updated repo
- Send to anthropic for skill analysis
 - Passes all the repo data to anthropic and ask it to return skills in json format
- Save to DB
 - Saves the githuburl and analyzes skills to github_profiles. If user is connected it updates the record without creating one with ON CONFLICT DO UPDATE

6.6 Job Controller

This controller manages all job related api endpoints. 4 Major functions:

- getJobs
 - Fetches a paginated, filterable list of jobs from the database
 - Supports pagination via LIMIT and OFFSET
 - Dynamically builds a parameterized SQL
 - Returns the row jobs plus pagination metadata
- getLatestJobs
 - Returns the most fetched batch of jobs
 - Delegates entirely to getLatestBatch() from jobService
 - Useful for showing fresh result
 - Redis cache is tried first and falls back to DB
- getJobsById
 - Fetches a single job by Id
 - Returns 404 if not matched
- triggerFetch

- Manually kicks off a job scraping/fetch operation on demand
- Calls fetchAndStoreJobs() to go fetch and persist new jobs
- Requires authenticated user(req.user) – logs who triggered it

Note: This is mostly for testing.

6.7 Matching Controller

This controller handles all job matching and application for the logged-in user. The 4 functions are:

- runMatching
 - runs matching all unmatched jobs for the logged-in user
 - fetches the user from db
 - uses the helper function buildUserPrefs
 - delegates to processAllJobsForUser()
 - returns a summary of what matched
- matchSingleJob
 - fetches both the user and job from db
 - runs processJobForUser from application service
 - returns application: null if jobs get skipped
 - matches one specific job against the logged-in user
- getMySkills
 - Returns the user's skills and match thresholds
 - Calls getUserSkills() from matchingService
 - Returns skills, count, experience level and both default + current thresholds
- getApplications
 - returns all applications for the logged-in user with optional filters
 - filters by status or is favorite
 - returns ordered by match_score DESC(descending order)
 - joins with jobs table to include job detail alongside each application

Note: Use of helper function BuildUserPrefs is used to accumulate and extract user preferences from the user row into clean object.

6.7 Resume Controller

This controller handles the uploading of resume along with skill extraction. The only function here is:]

- uploadResume
 - Uploads a PDF resume, extract skills using claude ai and save to db
 - Validates the file if it was uploaded, else returns 400 error
 - Extracts text from PDF using pdf-parse library
 - Send to anthropic for ai to extract skills
 - Prompt it to return only JSON array of skills no extra text
 - Save to db with filename, raw text and skills

Note: Anthropic is being used as a parser not a generator and skills come back as plain json array which is stored in PostgreSQL.

7. Services

The service folder consists of all the services or the heavy lifting called upon by the controller. In layman's term in a corporate environment(Our Application) there are managers(Controller) and then all the task performed by workers(Services). There are majorly 12 services working in our application to run our app smoothly. They are:

1. Application Engine Service
2. Application Service
3. Apply Service
4. Document Service
5. Job Service
6. Matching Service
7. Notification Service
8. Pdf Service
9. Pipeline Service
10. Playwright Agent

7.1 Application Engine Service

This service is the core automation engine of the app. It runs the full job application pipeline for the users. Core 5 functions are listed below:

- isUserEligible
 - mainly checks before doing anything like if user is inactive or already employ so kill switch initiated
- runPipelineForUser(main function)
 - fetches the user from db
 - checks the killswitch via the above function(isUserEligible)
 - match all unprocessed jobs via processAlljobsforuser
 - find pending applications that don't have documents yet
 - generate resume + cover letter for each via generateDocuments
 - mark them as applied or if fails push it to retry queue
- pushToRetryQueue
 - failed applications don't die, gets pushed to redis queue to be retried later
- processRetryQueue
 - works through the retry queue
 - keeps trying up to 6 time(MAX) after that permanently deems failed
- updateUserStatus
 - updates the user's jobs status and toggles kill switch accordingly

7.2 Application Service

This service handles deciding whether a user should apply to a job and save that result. This is mostly used by our application engine. Major 2 functions are:

- processJobForUser – handles one job for one user
 - checks if the user already applied to this job recently(default to 60 days)

- runs matching via `matchJobtoUser` to get score and decision
- saves the result to application with status
- uses upsert – if the application exists already just updates instead of duplicating
- `processAllJobsForUser` - handles multiple unmatched jobs for user
 - Queries jobs the user hasn't been matched against yet and filtered by their country
 - Pulls up to 200 jobs at a time
 - Loops through each one calling `processJobForUser`
 - Returns a summary of applied, skipped and favourites

7.3 Apply Service

This is the service that handles the actual act of applying to a job by generating documents and submitting the application and notifying the user. Most complex service in the application. Contains 3 functions which are:

- `processApplication`
 - runs the full apply pipeline for single job
 - loads the jobs and user from db
 - check if applied or skipped
 - generates tailored resume + cover letter
 - attempts to auto-apply via `playwright`(browser automation)
 - flags manual required if captcha or bot wall is detected
 - sends an email notification to the user
 - cleans up the temp resume file after everything is done
- `processAllMatched`
 - process bulk instead of single application
 - only picks jobs score higher than 55(which is default), up to 20 at a time
 - loops through each calling `processApplication`
 - adds random delay of 2 to 4 second between each job so as not to make it look like bot
 - supports SSE callback to stream live progress to the frontend
 - sends a weekly digest email at the end summarizing everything
- `markManuallyApplied`
 - when user manually applies it changes the status to manually applied

7.4 Document Service

This service is responsible for generating tailored AI resumes and cover letters for each job application. It uses `anthropic` to write documents specific to each job. Mostly 4 functions all to generate documents.

- `getGenerateContext(helper)`
 - this fetches everything needed before generating any documents in one go using `Promise.all`
 - user, job, baseresume, github profile, existing application all pulled simultaneously
- `generateResume`
 - generates tailored resume for specific jobs
 - combines skills from uploaded cv and github profile

- sends everything to anthropic with detailed prompt instruction to tailor resume to the job
- uses two pass system where first pass generates content and second pass corrects overflow
- saves the result as text and pdf to the db using upsert
- generateCoverLetter
 - generates tailored cover letter
 - similar to resume with different prompt
 - uses word count loop that runs 3 passes to land it between 245 to 275 word trying to hit that 260 sweet spot
 - saves the result similar to resume
- generateDocuments
 - main entry point called by other services
 - runs both generateResume and generateCoverLetter in parallel using promise. All
 - returns both together

7.5 Job Service

This service is responsible for fetching jobs from multiple sources, deduplicating, storing them, and caching the results. It's the data collection engine of the app. 3 core functions:

- fetchAndStoreJobs
 - runs the full job fetching pipeline
 - delegates to fetchRealJobs script which handles Adzuna + JSearch (OpenWebNinja)
 - caches the latest job IDs in Redis for 6 hours
 - saves new job to DB
- getLatestBatch
 - retrieves the most recently fetched jobs
 - tries redis first – if cached fetches the job rows by those id
 - if cache is cold(miss), falls back to querying from db directly
- pushToRetryQueue
 - pushes failed job application into redis retry queue for later

7.6 Matching Service

This is the brain of the matching system. It decides how well a user fits a job by scoring it across multiple dimensions and then verifying with AI. It has 2 main function.

- getThresholds
 - returns score thresholds based on experience level
 - Entry minimum: 35, default: 50, favourite: 70
 - Mid minimum: 45, default: 60, favourite: 78
 - Senior minimum: 50, default: 68, favourite: 83
- getUserSkills
 - pulls skill from both the user's cv and github profile and combines them and removes duplicates

Pre-filter

Runs before any scoring to eliminate obviously irrelevant jobs and save computation and api credits. Always skips completely unrelated fields such as devOps, security, design and checks for rescue signal such as mentorship, willingness to learn etc. and overrides everything. If no rescue signal skills title like senior, leader, and also 2+ years of experience for entry level users

Skills	Weight	What it Scores?
Skills	35 pts	How many skills the user has, with synonymous matching
Title	20 pts	How well the job matches user's target titles
Experience	15 pts	Gap between user's level and job required level
Location	12 pts	Remote, city match
Freshness	8 pts	How recently was the job posted
Company	5 pts	Preferred/bloked companies -10 if blocked

After the scoring anthropic is called for semantic skill score and overall skill score returns us matched skills, missing skills and reasoning

Final = (rule-based*70%) + (AI score * 30%)

- has AppliedRecently
 - checks for duplicate if the user applied within 60 days window period then returns true

7.7 Notification Service

This service handles all email notification sent to the user about their job application.

- sendMail(helper)
 - the base helper function which wraps the resend api call from, to subject with html and optional attachments
- verifyConnection(helper)
 - checks that resend api key exists on startup
- sendAppliedEmail
 - sent when job successfully auto-applied to
 - includes title, email, company etc.
- sendManualRequiredEmail
 - sent when playwright hits captcha or botwall
 - includes same job details and reason of failure
 - attaches auto generated resume and cover letter as pdf so user can apply themselves
- sendFailedEmail
 - sent when auto apply crashes
 - includes error message so user know what is wrong
- sendWeeklyDigest
 - sent at the end summarizing everything
 - groups in 3 section – auto applied, manual required and failed

7.8 Pdf Service

This service handles converting text content into professional format PDF files for both resume and cover letter. We use PDFKIT to draw everything manually.

- `GenerateResumePDF`
 - Renders everything and splits section into dynamic and fixed
 - Returns pdf buffer plus fill ratio metrics
- `GenerateCoverLetterPDF`
 - Renders name from to and divides the body into 3 paragraphs
- `parseResumeSections(helper)` – takes the raw text from AI and splits it into structured section
- `estimateSectionHeight(helper)` – calculates how tall a section will be before rendering it
- `drawSectionHeader(helper)` – draws bold section title with horizontal line underneath
- `renderline(helper)` – handles rendering a single line differently depending on what type it is

7.9 Pipeline Service

The service is the orchestrator and ties every service together and runs the full end to end pipeline in sequence.

- `Run`
 - Executes the full pipeline in 3 stages for user
 - Fetch - calls `fetchRealJobs` to get fresh job listing from all sources
 - Match – builds user preference and runs `processAllJobsForUser` to score new jobs
 - Apply – calls `processAllMatched` to generate documents and apply to all jobs that pass threshold
 - At every stage it emits a `send()` event so frontend can show user live progress to the user
- `getLatestResult`
 - fetches the most recently processed application from the last 24 hour display on the dashboard.

7.10 Playwright Agent

This service is responsible for automating the actual job application. Opens a real browser and navigates job posting and fills the form.

- `randomDelay(Helper)` — adds a random pause between actions to mimic human behaviour and avoid bot detection.
- `humanType(Helper)` — types text character by character with random delays instead of pasting it all at once — makes it look like a real person typing.
- `humanClick(Helper)` — moves the mouse naturally to a random point within a button before clicking, rather than clicking the exact centre every time.
- `detectBotProtection(Helper)` — scans the page for signs of bot protection before and after filling the form. Checks for reCAPTCHA, hCaptcha, Cloudflare Turnstile, login walls, ATS platforms (Workday, Greenhouse, Lever), and modal overlays.

- `fillApplicationForm(Helper)` — intelligently finds and fills form fields by searching for common selectors for name, email, phone, LinkedIn, cover letter and resume upload. Skips fields it can't find rather than crashing.
- `submitForm(Helper)` — finds and clicks the submit button by trying standard selectors first, then falling back to text-based button detection.
- `detectSuccess(Helper)` — after submitting, checks the page for confirmation text like "application submitted" or "thank you for applying" to verify it actually worked.
- `applyToJob(function)`
 - launches headless browser with real user agent and Canadian locale to avoid detection
 - clicks the apply button first before filling the form
 - navigates to job url
 - checks for bot protection
 - fills the form
 - rechecks for bot protection after filling
 - submits the form
 - returns `auto_applied` or `manual required`

8. Middleware

Middleware sits between the request coming in and the route handler that processes it. Its like a checkpoint which passes through before it gets to the controller.

1. Auth Middleware
2. Rate Limiter Middleware

8.1 Auth Middleware

This is a JWT authentication guard. This protects our routes that require a logged in user.

- Extracts the token from Authorization header – strips the bearer prefix
- Rejects immediately with 401 if no token is present
- Verifies the token against your `JWT_SECRET` – if its valid attaches the decoded user into `req.user` and calls `next()` to continue else returns 401

8.2 Rate Limiter Middleware

This middleware prevents abuse by limiting how many requests a client can make in a given time window. It has 3 limiters each with different strictness depending on the route.

- `apiLimiter` – general protection for all api routes
 - 100 requests per 15 minutes
 - Applied broadly across the app
- `authLimiter` – strict protection for login/register routes
 - only 10 attempts per 15 minutes
 - prevents brute force attacks on passwords
- `pipelineLimiter`
 - only 5 runs per hour

- prevents someone from spamming the pipeline which would burn Anthropic api credits and hammer job sites

9. Scripts

This is not part of the main app flow and can run independently without cron or anything. This just contains fetchRealJobs.

9.1 FetchRealJobs

- Reads the country from active user in the db
- Fetches from adzuna which runs 9 different search queries sequentially with 1 second delay between each
- Fetches from JSearch(OpenWebNinja) which runs 8 queries pulling from linkedin, indeed, Glassdoor and google jobs
- Deduplicates removes the duplicates before saving
- Filter filters out anything older than 2 days
- Saves to db upserts skips existing job
- Print summary prints the summary of the jobs latest 10 jobs

10. Routes

This folder defines all my api endpoints. Think of it as a traffic basically you're in a car trying to go somewhere and the traffic is going to control you or guide you where to go. Without routes our controller and logic have no idea what to do. There are 9 routes which are listed below:

1. Application Engine Routes
2. Apply Routes
3. Auth Routes
4. Document Routes
5. GitHub Routes
6. Job Routes
7. Matching Routes
8. Pipeline Routes
9. Resume Routes

10.1 Application Engine Route

Method	Endpoint	Controller	Purpose
POST	/api/engine/run	runPipeline	Triggers the full pipeline for the logged in user
POST	/api/engine/retry	runRetryQueue	Processes any failed applications in the retry queue
POST	/api/engine/status	setStatus	Updates the user's job search status and toggles kill switch

10.2 Apply Routes

Method	Endpoint	Purpose
--------	----------	---------

POST	/api/apply/run	Waits for the result and returns summary
POST	/api/apply/single	Applies to a single specific job by JobId
PATCH	/api/apply/:applicationId/manual	Marks a manual_required application as manually_applied
POST	/api/apply/test-email	Smoke test – sends a test email to verify Resend is working

10.3 Auth Routes

Method	Endpoint	Controller	Purpose
POST	/api/auth/register	authLimiter	Register a new user
POST	/api/auth/login	authLimiter	Login and receive access + refresh token
POST	/api/auth/logout	None	Logout and clear refresh token
POST	/api/auth/refresh	None	Get a new access token using refresh token
GET	/api/auth/profile	Auth	Get logged in user's profile
PUT	/api/auth/profile	Auth	Update logged in user's profile

10.4 Document Routes

Method	Endpoint	Purpose
POST	/api/documents/resume/:jobId	Generate a tailored resume for a specific job
POST	/api/documents/cover-letter/:jobId	Generate a tailored cover letter for a specific job
POST	/api/documents/generate/:jobId	Generate both resume and cover letter in one call
GET	/api/documents/resume/:jobId/download	Download the generated resume as a PDF
GET	/api/documents/cover-letter/:jobId/download	Download the generated cover letter as a PDF

Note: JobId is tied to specific job so we can't generate generic resume.

10.5 GitHub Routes

Method	Endpoint	Middleware	Purpose
POST	/api/github/connect	auth	Connect a GitHub profile, fetch repos and extract skills via AI

10.6 Job Routes

Method	Endpoint	Purpose
GET	/api/jobs	Get paginated and filtered list of jobs
GET	/api/jobs/latest	Get the most recently fetched batch from Redis cache
POST	/api/jobs/trigger	Manually trigger a job fetch – mostly for testing
GET	/api/jobs/:id	Get a single job by ID

10.7 Matching Routes

Method	Endpoint	Purpose
POST	/api/matching/run	Run matching against all unmatched jobs for the logged in user
POST	/api/matching/job/:jobId	Match a single specific job against the logged in user
GET	/api/matching/skills	Get the logged in user's skills and match thresholds
GET	/api/matching/applications	Get all applications for the logged in user with optional filters

10.8 Pipeline Routes

Method	Endpoint	Purpose
POST	/api/pipeline/run	Runs the full pipelines – fetch -> match -> generate -> apply. Streams live progress to the frontend via SSE
GET	/api/pipeline/results	Returns the latest pipeline results for the logged in user

10.9 Resume Routes

Method	Endpoint	Middleware	Purpose
POST	/api/resume/upload	Auth + multer	Upload a PDF resume, extract skills via AI and save to DB

11. Client

The frontend is built with React + Vite + Tailwind Css and serves as the dashboard for the users to interact with the application.

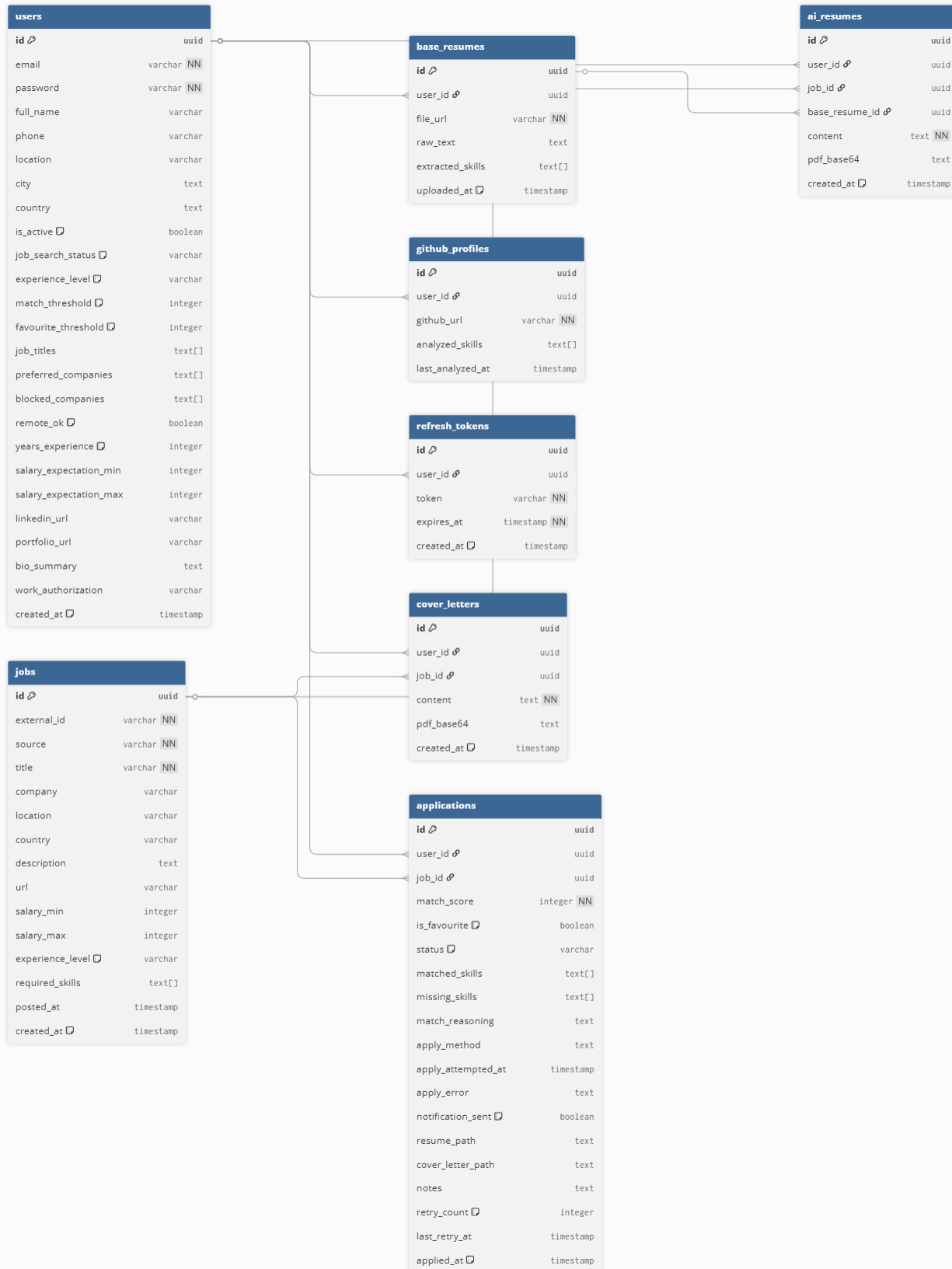
Key Pages/Views:

- Dashboard: shows recent application vs manual required, shows pipeline
- Applications – view all matched applications with scores, matched skills and status
- Profile – manage user details, experience level, preferred companies, and links
- Upload CV - upload base

All API calls go through the Express backend via REST endpoints. Pipeline progress is streamed live using **SSE** so the dashboard updates in real time as jobs are fetched, matched and applied to JWT access token is stored and sent with every request for authentication. Refresh token is stored as an **httpOnly cookie** and used to silently renew the access token.

11. Database

The application uses PostgreSQL as its primary relational database, managed through 14 sequential migration files that build and evolve the schema incrementally. There are 8 core tables connected through UUID foreign keys with cascade deletes, meaning if a user is deleted, all their resumes, applications, github profiles and tokens are automatically removed. The schema is designed around the user as the central entity, with jobs as an independent feed that gets matched and applied to per user. Redis sits alongside PostgreSQL handling job caching (6 hour TTL-TIME TO LIVE), the retry queue for failed applications, and rate limiting. It is not persisted to disk and is treated as a fast temporary store only.



12. Key Design Decision

12.1 Weighted Scoring over Strict Threshold

The original system matched a user's GitHub and base CV skills directly against a job's required skills using a strict 70% threshold. Any job below that threshold was discarded regardless of other factors. This produced low return — a high volume of jobs fetched but very few passed the filter.

The system was replaced with a weighted multi-dimension scoring engine that evaluates each job across 6 dimensions: skills, title alignment, experience level, location, job freshness, and company preference. Each dimension carries a different weight reflecting its importance, with skills weighted heaviest at 35 points.

Once the rule-based dimensions are scored, the result is passed to Anthropic Claude as a pre-score. Claude evaluates the job description against the user's skills semantically — meaning it understands that Node.js implies Express familiarity, that React and React.js are the same technology, and awards partial credit for transferable skills that exact keyword matching would miss. Claude returns a semantic skill score (0–5) and an overall fit score (0–100).

The final score is calculated as:

Final Score = (Rule-based Score × 70%) + (Claude Overall Score × 30%)

The 70/30 split ensures the rule-based system always anchors the result — Claude can adjust the score up or down based on semantic understanding but cannot override a clearly poor match. If Claude's API is unavailable, the system falls back to rule-based scoring only and continues operating without interruption.

12.2 Jsearch over multiple free api

Before we used Indeed RSS, Jooble, JobBank and ArbeitNow which were either blocking the api or were not giving better coverage. Hence, we opted for OpenWebNinja which aggregates job posting from linkedin, indeed, glassdoors and google jobs in real time which are one of the largest job aggregators.

12.3 Two-Pass PDF System

The resume generator runs Anthropic in two passes. The first pass generates the content. The second pass measures how much of the page is filled and either trims or expands the bullets to hit exactly one page. This ensures a clean professional output everytime without manual editing though it does cost little api credits.

12.4 SSE for Live Pipeline Progress

Instead of making the user wait for the pipeline to finish with no feedback, Server-Sent Events stream live progress to the dashboard as each stage completes like fetch, match, apply so the user sees jobs being processed in real time.

12.5 Redis Retry Queue

Failed applications are not lost. They are pushed to a Redis queue and retries up to 6 times automatically. Only after 6 failures is an application permanently marked as failed. This handles temporary network errors and site downtime without losing applications.

12.6 Separate JWT Secrets

Access Tokens and refresh tokens use different secrets. This means a compromised access token cannot be used to forge a refresh token, keeping long-lived sessions secure.

12.7 Semi-Automated Playwright

Fully automated applying is not possible on most jobsites due to CAPTCHAs, login wall, bot detection and ATS platforms. Semi automated means if it doesn't detect anything it will apply but if not it will flag them as manual required send a email with generated resume and cover letter so the user get's live feedback on what to apply with and get's the first advantage.

12.8 PostgreSQL Vs. MongoDB

MongoDB is a document database as it stores in JSON objects with no fixed structure. That works well when our data has no relationship but our data is highly relational.

- User has resume
- Resume links to an application
- Application has links to job
- Job links to ai resume and cover letter
- Both cover letter and resume link back to job

If we went with mongodb we'd either end up duplicating data losing data integrity and making queries slow and complex. PostgreSQL handles all of that natively with foreign keys and cascade, JOIN queries.

PostgreSQL was chosen over SQLite and MySQL because SQLite is too lightweight for production, also it has no proper enum support. MySQL is viable but PostgreSQL has better native support for array types, UUIDs and enum types which this application uses extensively throughout the schema.

References

AI Job Automation Tool — Technical Document and Developer Guide

1. Backend & Runtime

- [1] Node.js — JavaScript runtime environment. <https://nodejs.org>
- [2] Express.js — Web framework for Node.js. <https://expressjs.com>
- [3] Cors — Cross-Origin Resource Sharing middleware. <https://github.com/expressjs/cors>
- [4] Multer — Middleware for handling multipart/form-data file uploads. <https://github.com/expressjs/multer>
- [5] Node-cron — Task scheduler using cron syntax for Node.js. <https://github.com/node-cron/node-cron>
- [6] Express-rate-limit — Rate limiting middleware for Express. <https://github.com/express-limit/express-rate-limit>
- [7] Winston — Logging library with multiple transports and log levels. <https://github.com/winstonjs/winston>
- [8] Axios — Promise-based HTTP client for Node.js and the browser. <https://axios-http.com>

2. Frontend

- [9] React — JavaScript library for building user interfaces. <https://react.dev>
- [10] Vite — Next generation frontend build tool. <https://vitejs.dev>
- [11] Tailwind CSS — Utility-first CSS framework. <https://tailwindcss.com>

3. Database & Caching

- [12] PostgreSQL — Open source relational database system. <https://www.postgresql.org>
- [13] Redis — In-memory data structure store used for caching and queuing. <https://redis.io>
- [14] Docker — Platform for containerizing and running applications. <https://www.docker.com>

4. Authentication & Security

- [15] JSON Web Token (jsonwebtoken) — JWT implementation for Node.js. <https://github.com/auth0/node-jsonwebtoken>
- [16] Bcrypt — Password hashing library for Node.js. <https://github.com/kelektiv/node.bcrypt.js>
- [17] JWT Introduction — Overview of JWT structure and best practices. <https://jwt.io/introduction>

5. PDF Processing

- [18] PDFKit — PDF generation library for Node.js. <https://pdfkit.org>

[19] pdf-parse — Extract text content from PDF files. <https://www.npmjs.com/package/pdf-parse>

6. Browser Automation

[20] Playwright — End-to-end browser automation framework by Microsoft. <https://playwright.dev>

7. External APIs & Services

[21] Anthropic Claude API — AI API used for job matching, resume generation and skill extraction. <https://docs.anthropic.com>

[22] Adzuna API — Job listing aggregator API. <https://developer.adzuna.com>

[23] OpenWebNinja (JSearch) — Job search API aggregating LinkedIn, Indeed, Glassdoor and Google Jobs. <https://rapidapi.com/openwebninja/api/jsearch>

[24] Resend — Email delivery API used for application notifications. <https://resend.com/docs>

8. Web Standards & Protocols

[25] Server-Sent Events (SSE) — MDN Web Docs — streaming live updates from server to client. https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events

[26] HTTP REST API Design — MDN Web Docs — HTTP methods and REST conventions. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>